

Spring Framework — XML-Based Configuration

Complete Study & Interview Notes

1. What is XML-Based Configuration?

Spring can receive Configuration Metadata through Annotation-Based Configuration or XML-Based Configuration.

Annotation: @Component, @Bean, @Configuration, @ComponentScan

XML: beans.xml, <bean>, <constructor-arg>, <property>

XML configuration is mainly encountered in legacy Spring applications, older enterprise systems, and hybrid applications during migration.

2. IoC Container and Configuration Metadata

The IoC Container creates, manages, injects dependencies into, and destroys Spring beans. It needs Configuration Metadata to know which beans to manage.

```
Configuration Metadata
|
+-----+
|       |
|       |
Annotations XML
|       |
@Configuration beans.xml
@Component <bean>
@Bean      <constructor-arg>
```

3. Annotation vs XML

```
// Annotation
@Component
public class OrderService { }
```

```
@Configuration
@ComponentScan
class AppConfig { }
```

```
new AnnotationConfigApplicationContext(AppConfig.class);
```

```
<!-- XML -->
<bean id="orderService" class="in.coders.OrderService"/>
```

```
new ClassPathXmlApplicationContext("beans.xml");
```

4. ClassPathXmlApplicationContext

ClassPathXmlApplicationContext loads XML configuration from the classpath.

```
ApplicationContext context =
    new ClassPathXmlApplicationContext("beans.xml");
```

5. Location of beans.xml

```
src/main/resources/beans.xml
```

The resources directory is placed on the classpath, allowing ClassPathXmlApplicationContext to find the file.

6. Basic Definition

```
<bean id="orderService"
      class="in.coders.OrderService"/>
```

id identifies the bean. class contains the fully qualified Java class name.

7. XML Does Not Change Bean Lifecycle

Read XML → Bean Definition → Instantiate → Dependency Injection

→ Initialization → Bean Ready → Use → Destruction

The configuration source changes, but the fundamental bean-management lifecycle remains the same.

8. Getting Beans

```
// By type
context.getBean(OrderService.class);

// By name / id
context.getBean("orderService");

// By name + type
context.getBean("orderService", OrderService.class);
```

By name returns Object and therefore needs casting. Name + type avoids explicit casting. By type can become ambiguous when multiple beans have the same type.

9. Multiple Beans of Same Type

```
<bean id="orderService" class="in.coders.OrderService"/>
<bean id="orderService2" class="in.coders.OrderService"/>
```

Both bean definitions can exist and create separate instances. `getBean(OrderService.class)` can become ambiguous and may throw `NoUniqueBeanDefinitionException`.

10. Bean id, name and aliases

```
<bean id="orderService"
      name="orderServiceBean orderService2 orderService3"
      class="in.coders.OrderService"/>
```

The id is the unique bean identifier. A bean can have multiple names/aliases that refer to the same bean.

11. Dependency Injection with XML

The transcript covers constructor injection and setter injection. Field injection is not configured through the XML bean-definition approach discussed.

```
public OrderService(PaymentService paymentService) {
    this.paymentService = paymentService;
}
```

12. Constructor Injection

```
<bean id="paymentService"
      class="in.coders.PaymentService"/>

<bean id="orderService"
      class="in.coders.OrderService">
    <constructor-arg ref="paymentService"/>
</bean>
```

ref means: find the Spring bean with that ID and pass its object to the constructor.

13. ref vs value

```
<constructor-arg ref="paymentService"/>

<constructor-arg value="UPI"/>
```

ref references another Spring bean. value provides a literal/simple value.

14. Multiple Constructor Arguments

```
public PaymentService(String type, int retryAccount) {
    this.type = type;
    this.retryAccount = retryAccount;
}

<constructor-arg value="UPI"/>
```

```
<constructor-arg value="3"/>
```

Constructor arguments can also be mapped explicitly using index or parameter name.

```
<constructor-arg index="0" value="UPI"/>
<constructor-arg index="1" value="3"/>
```

```
<constructor-arg name="type" value="UPI"/>
<constructor-arg name="retryAccount" value="3"/>
```

15. Dependency Resolution Flow

```
Read paymentService definition
↓
Create PaymentService
↓
Read orderService definition
↓
See constructor dependency
↓
Find paymentService bean
↓
Pass it to OrderService constructor
↓
Create OrderService
```

16. Initialization and Destruction Methods

```
public void init() { }
public void cleanup() { }
```

```
<bean id="userService"
      class="in.coders.UserService"
      init-method="init"
      destroy-method="cleanup"/>
```

init-method specifies the initialization callback. destroy-method specifies the destruction callback.

17. Closing XML Context

```
ClassPathXmlApplicationContext context =
    new ClassPathXmlApplicationContext("beans.xml");

context.close();
```

Closing the context allows destruction callbacks for managed singleton beans.

18. Prototype Scope

```
<bean id="userService"
      class="in.coders.UserService"
      scope="prototype"/>
```

Prototype beans receive a new instance when requested, but Spring does not manage their complete destruction lifecycle in the same way as singleton beans. Application code must handle cleanup of resources owned by prototype objects.

19. Splitting XML Configuration

Large XML files can be divided into multiple configuration files, for example beans.xml and beans2.xml, to keep related configuration manageable.

20. Importing XML Files

```
<import resource="beans.xml"/>
<import resource="beans2.xml"/>
```

A parent configuration can import multiple XML configuration files.

21. XML + Annotation Hybrid

Legacy systems can contain XML-defined beans and annotation-defined components in the same IoC container. This is useful during gradual migration from XML to annotation-based configuration.

22. Important XML Elements

<code><bean></code>	→ bean definition
<code>id</code>	→ unique identifier
<code>name</code>	→ name / aliases
<code>class</code>	→ fully qualified class
<code><constructor-arg></code>	→ constructor injection
<code>ref</code>	→ reference to another bean
<code>value</code>	→ literal value
<code>index</code>	→ constructor argument position
<code>init-method</code>	→ initialization method
<code>destroy-method</code>	→ destruction method
<code><import></code>	→ import another XML file

23. Advantages and Disadvantages

Advantage: Spring configuration can stay outside Java classes, keeping classes relatively free of Spring-specific configuration annotations.

Disadvantages: XML can become verbose, large, complicated, and harder to maintain. Modern Spring Boot development generally favors annotation/Java-based configuration.

24. Complete Example

```
// PaymentService.java
public class PaymentService {
    private String type;
    private int retryAccount;

    public PaymentService(String type, int retryAccount) {
        this.type = type;
        this.retryAccount = retryAccount;
    }

    public void pay() {
        System.out.println("Payment Done using " + type);
    }
}

// OrderService.java
public class OrderService {
    private PaymentService paymentService;

    public OrderService(PaymentService paymentService) {
        this.paymentService = paymentService;
    }

    public void placeOrder() {
        paymentService.pay();
        System.out.println("Order Placed");
    }
}

<!-- beans.xml -->
<bean id="paymentService" class="in.coders.PaymentService">
    <constructor-arg value="UPI"/>
    <constructor-arg value="3"/>
</bean>

<bean id="orderService" class="in.coders.OrderService">
    <constructor-arg ref="paymentService"/>
</bean>

ClassPathXmlApplicationContext context =
    new ClassPathXmlApplicationContext("beans.xml");

OrderService orderService =
    context.getBean("orderService", OrderService.class);

orderService.placeOrder();
```

```
context.close();
```

25. Interview Questions

Q: What is XML-based configuration?

A: A way of defining Spring bean metadata and dependencies in an XML file.

Q: Which class loads XML configuration?

A: `ClassPathXmlApplicationContext`.

Q: Where is `beans.xml` generally placed?

A: `src/main/resources`.

Q: What does `<bean>` represent?

A: A Spring bean definition.

Q: `ref` vs `value`?

A: `ref` references another Spring bean; `value` supplies a literal value.

Q: How is constructor injection done?

A: Using `<constructor-arg>`.

Q: Can multiple beans have the same class?

A: Yes, with distinct bean identities.

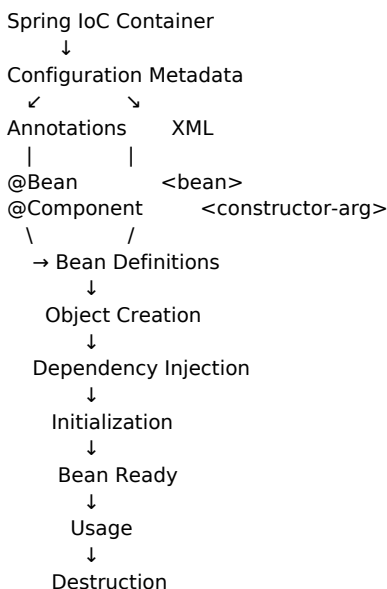
Q: What if type lookup finds multiple beans?

A: The lookup can become ambiguous and throw `NoUniqueBeanDefinitionException`.

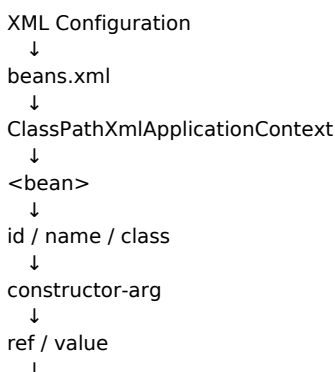
Q: Can XML and annotations be used together?

A: Yes, in a hybrid configuration.

26. Final Mental Model



27. What You Actually Need to Remember



index / name
↓
init-method / destroy-method
↓
import
↓
XML + Annotation Hybrid

Focus on the concepts rather than memorizing the complete XML schema/header.

28. One-Line Summary

XML-based configuration is another way of providing Bean Definition Metadata to the Spring IoC Container; the underlying bean-management and lifecycle concepts remain the same.